Nightfall Labs
Team Deliverable 4 – Project Phase II
ITIS 3300 – Spring 2026

Team Members:
Danny Moncada
Thomas Clayton
Alex Lopez Robles
Jayant Thathireddy
Isaac Baraki
Justus Purnell

==========

A. REQUIREMENTS

==========\

<u>Brief Phase II Scope Summary:</u>
This phase focuses more on implementation, integration, and testing of the restaurant ordering system. In this phase, the team focuses on making the main user flow functional across the frontend and backend, including browsing menu items, applying filters, managing the cart, validating checkout information, submitting orders, storing order data, and viewing confirmation and order history. The overall scope did not significantly change, but the requirements are now written in more detail to reflect the current working version of the system.

<u>If scope changed:</u>
The scope did not significantly change. The Phase II work builds on the same plan and expands it into a more complete system. The main update is that the requirements are now described with more detail based on the features currently implemented in the frontend, backend, and database.

Functional Requirements for Phase II:

FR1: Menu Browsing and Filtering
Description: The system allows users to browse available menu items and narrow results using category filters, allergen exclusions, dietary restriction filters, and a search feature so users can more easily find items that match their preferences.

FR2: Cart Management
Description: The system allows users to add items to the cart, update item quantities, remove items, and view the current subtotal, tax, delivery fee, service fee, tip, and final total before checkout.

FR3: Checkout Validation
Description: The system allows users to enter checkout information, including order type, email, cardholder name, card number, and delivery address when needed, and validates required fields before the order can be submitted.

FR4: Order Submission and Storage
Description: The system allows completed order information to be sent from the frontend to the backend and stored in the database so the order can be processed and retrieved later.

FR5: Confirmation and Order History
Description: The system allows users to view a confirmation page after a successful order is placed and displays the generated order ID. The system also allows users to view previously placed orders in the order history page.

Non-Functional Requirements for Phase II:

NFR1: Usability

Description: The system should provide a simple and organized user experience with clear pages for menu browsing, cart review, checkout, confirmation, and order history.

NFR2: Maintainability

Description: The system should be organized into separate frontend and backend components with modular files so future updates, debugging, and feature improvements can be completed more easily.

NFR3: Reliability

Description: The system should reliably process valid user input, prevent incomplete checkout submissions, and return stored order information without crashing during normal use.

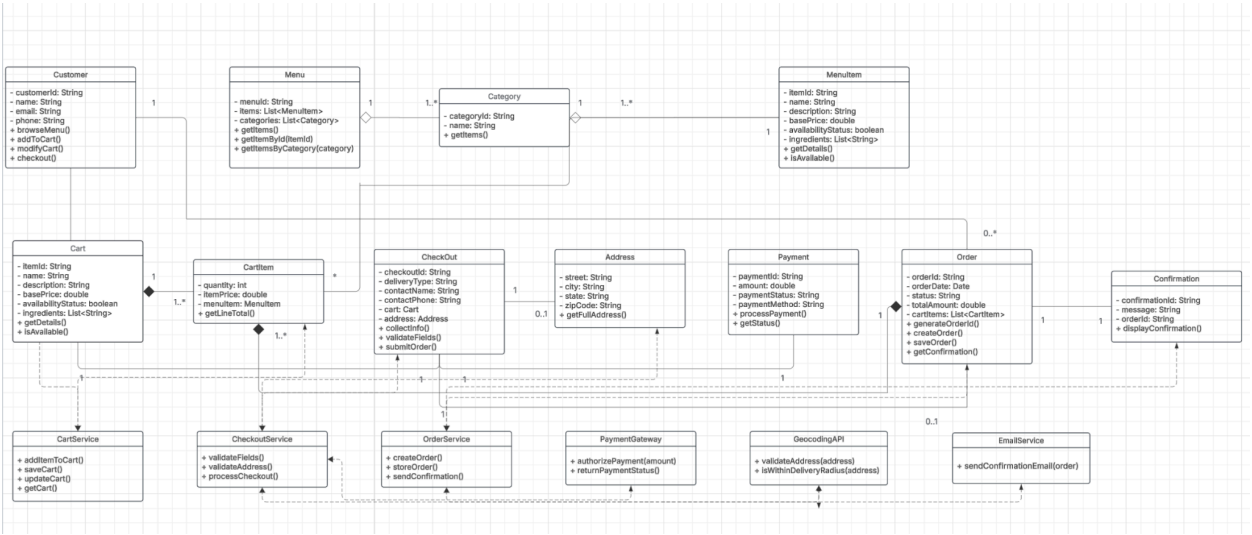
==========

B. UML DESIGN

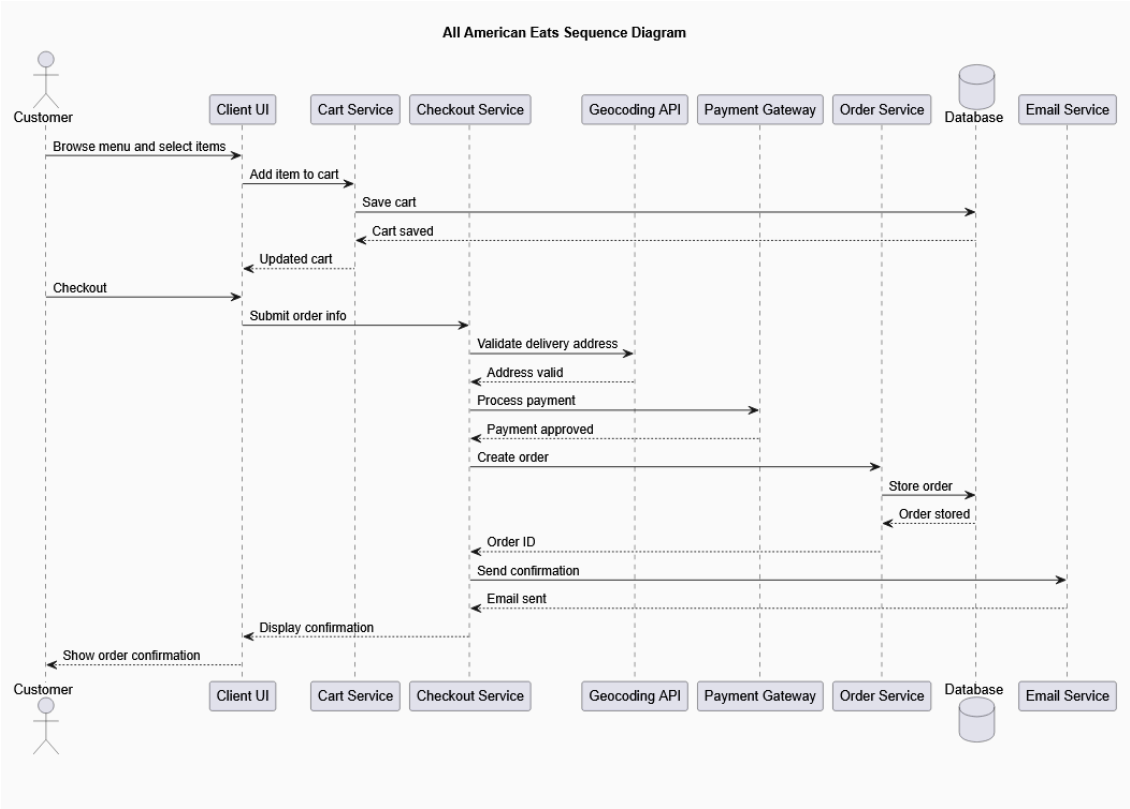
==========

You must include the following diagrams:

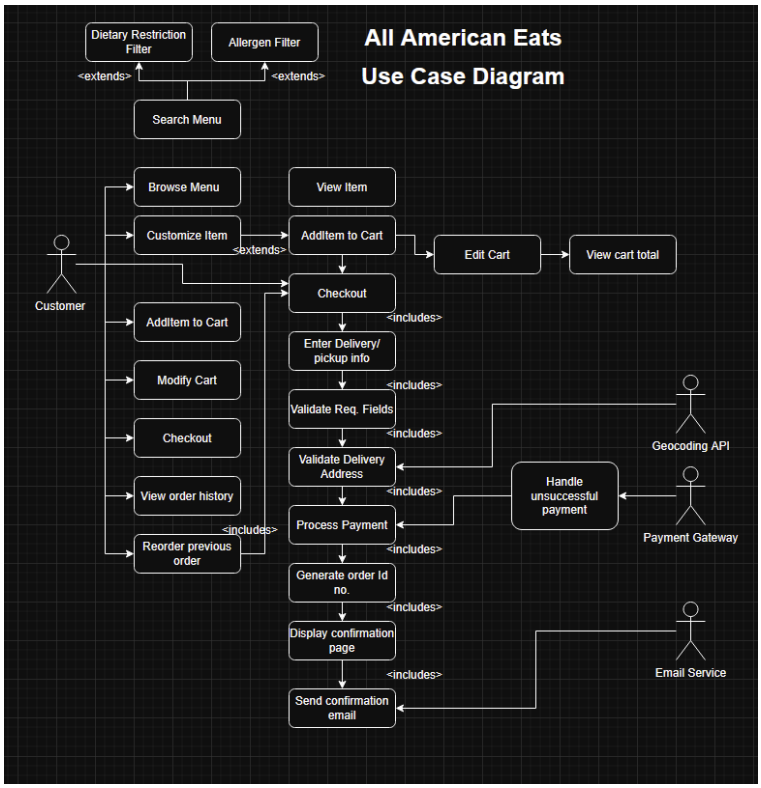
- **Class diagram**



• Sequence diagram



• Use case diagram



==========

C. USER MANUAL

==========

OVERVIEW:
Welcome to the All American Eats ordering application! This system allows users to browse our delicious menu, filter items based on their preferences, add selections to a cart, and place an order for delivery or pickup.

INSTALLATION (End User instructions):
For end-users, there is no installation required. Simply open your modern web browser (Google Chrome, Firefox, Edge) and navigate to the application URL provided by the restaurant (locally, this will be your `index.html` file).

HOW TO USE:
1. Browsing the Menu:
   - Navigate to the Home page ('Menu' tab).
   - Use the category filters on the left sidebar to show only specific types of items (such as Burgers, Wraps, or Salads).
   - Use the dietary restrictions checkboxes or allergen exclusions to filter out unwanted items.
   - Use the top search bar to look for a specific craving you have.
2. Adding to Cart:
   - Click on an item to view its details.
   - Add it to your cart (and select any customizations if applicable).
3. Viewing your Cart:
   - Navigate to the 'Cart' page using the top navigation bar.
   - Here you can review your items, subtotal, tax, fees, and adjust your tip.
4. Checkout:
   - Proceed to checkout from the cart page.
   - Select whether you want "Delivery" or "Pickup".
   - Enter your email, cardholder name, and 16-digit card number.
   - If "Delivery" is selected, enter your delivery address.
   - Click "Place Order" to finalize your purchase!
5. Viewing Past Orders:

- Navigate to the 'Orders' page in the top menu to view your localized order history and totals.

==========

D. INSTRUCTIONS TO COMPILE/RUN THE PROGRAM AND TEST CASES

==========

SYSTEM PREREQUISITES:
- Node.js (version 14.x or higher) installed. This allows the backend server to run.
- NPM (Node Package Manager).

COMPILING & RUNNING THE APPLICATION:
1. Setting up the Backend:
   - Open a command line interface (Terminal/Command Prompt) in the root directory of the project.
   - Run the following command to download the server dependencies (reads from package.json):
     `npm install`
   - Change directories to the backend folder and start the application server:
     `cd src/backend`
     `node server.js`
   - You should see the message: "Connected to database" and "Server running on http://localhost:3000". Leave this terminal window open.

2. Running the Frontend:
   - Open the directory `src/frontend` in your file explorer.
   - Double-click on `index.html` to open the application in your default web browser.
   - Ensure the server is actively running in the background for database interactions (like adding cart items or retrieving orders) to succeed.

RUNNING TEST CASES:
Currently, the application relies on manual acceptance testing to verify functionality.
- Open the application in your browser (`index.html`).

- Perform the following manual test scenarios to verify functionality:
  - Test 1 (Menu filters): Check all dietary checkboxes and confirm only compliant menu items appear.
  - Test 2 (Cart Validation): Attempt to proceed to checkout with an empty cart (the application should prevent this).
  - Test 3 (Checkout form): Enter an invalid credit card number during checkout and verify that it requires exactly 16 digits.
  - Test 4 (Order lifecycle): Complete a successful order, go to the "Orders" page, and ensure the new order ID correctly displays alongside the correct total.

==========

E. TEST CASES

==========

Functionality/ Method	Input	Output	Pass/Fail
Menu Filtering	Select "Vegan" filter	Only vegan menu items displayed	Pass
Menu Search	Search "Burger" in search bar	Only burger-related items appear	Pass
Add to Cart	Click "Add to Cart" on item	Item appears in cart with correct quantity	Pass
Update Quantity	Increase item quantity to 2	Cart updates total and quantity correctly	Pass
Remove Item	Click remove button on item	Item is removed from cart	Pass

Empty Cart Checkout	Attempt checkout with empty cart	Error message prevents checkout	Pass
Checkout Validation	Leave email field blank	Error message shown	Pass
Invalid Card Input	Enter card number "1234"	Error: must be 16 digits	Pass
Delivery Option	Select delivery without address	Error message shown	Pass
Successful Order	Enter valid checkout info and submit	Confirmation page with order ID displayed	Pass
Order Storage	Complete an order	Order appears in order history page	Pass
Order History	Navigate to orders page	Previous orders displayed correctly	Pass

==========

F. CODE INSPECTION FEEDBACK

==========

<u>Summary of Feedback Received:</u>

During the code inspection session, the team reviewed the system for logic errors, validation, and overall functionality. Most areas of the code were working correctly with no major issues found. The feedback mainly focused on improving consistency between different parts of the system, especially between the cart and checkout processes.

Main Issues Identified:
- Tip value was hardcoded in cart.js while users enter a tip during checkout
- Delivery fee and service fee were calculated in cart.js but not properly reflected in checkout.js
- Minor inconsistencies in how values such as tax and fees were handled across files

Actions Taken Based on Feedback:
- Updated the tip logic to ensure the user-entered tip in checkout is correctly used instead of a hardcoded value
- Fixed the integration between cart.js and checkout.js so delivery and service fees are consistently displayed and calculated
- Reviewed and aligned how numerical values such as tax and fees are handled across the system for consistency
- No major structural issues were found, but future improvements include refining data flow between components

==========

G. REFLECTION

==========

What Has Been Accomplished:
In this phase, we successfully turned our design into a working system where users can browse menu items, filter options, add items to a cart, and complete orders through checkout. The system now connects the frontend and backend, allowing orders to be stored and displayed in the order history. Overall, the main features of the application are fully functional and meet the requirements for this phase.

What Went Well:
The integration between the frontend and backend went well, especially with cart functionality and order submission. The team worked efficiently by dividing tasks, which helped us complete different parts of the system at the

same time. Most of the features worked as expected, and testing showed that the system handles normal user actions correctly.

What Could Be Improved:
One area that could be improved is better consistency between different parts of the system, such as ensuring values like tips and fees are handled the same way across all files. In addition, adding more automated testing instead of relying mainly on manual testing would make the system more reliable. Improving small UI details and handling more edge cases would also make the application stronger.

==========

H. MEMBER CONTRIBUTION TABLE

==========

Member Name	Contribution Description	Overall Contribution (%)	Note
Danny Moncada	Wrote Part A. Requirements for Deliverable 4 and started the final submission document for the team.	20%	
Thomas Clayton	-(Part B) Created the use case diagram. -(Part B) Created the sequence diagram. -Document Formatting fixes (broken tables from document conversion)	20%	
Alex Lopez Robles	Completed Parts E, F, and G of the report. - Designed test cases and documented code	20%	

	inspection feedback - Wrote the project reflection		
Jayant Thathireddy	- Designed and implemented frontend styling - Fixed bugs found during testing, created a better user experience - Updated business logic and implemented a consistent user flow	20%	
Isaac Baraki	- Looked over the report to make sure everything was accurate and matched the actual system - Helped catch inconsistencies and improve clarity -Assisted with final review before submission	20%	
Justus Purnell			